

NUMERICAL METHODS (ENSH 202)

Micro Syllabus

Lecture : 3
Tutorial : 1
Practical : 3

Year : II
Part : I

Unit 1: Solution of Non-Linear Equations (7 hours)

Sub Unit		Further Details	Formula Derivation	Working Mechanism	Algorithm	Numerical	Comparison with	Advantages/ Limitations
1.1	Errors and accuracy in numerical computations	- Accuracy Vs. Precision - Sources or types of error - Truncation, rounding-off, significant figures - Error Calculation: absolute, relative, % errors	N/A	N/A	N/A	Yes	Each other	---
1.2	Bisection Method		No. of Iterations	Yes	Yes	Yes	Trial and Error	Yes
1.3	Regula Falsi Method and Secant Method							
	Regula Falsi Method		Graphically	Yes	Yes	Yes	Bisection Method	Yes
	Secant Method		Graphically	Yes	Yes	Yes	Regula Falsi	Yes
1.4	Newton Raphson Method	Only for real roots	Graphically & Analytically	Yes	Yes	Yes	Secant Method	Yes
1.6	Comparison of the methods	Bracketing Vs. Open-ended methods	N/A	N/A	N/A	N/A	Each other	Yes
		Rates of Convergence	Bisection & Newton-Raphson	N/A	N/A	N/A	Each other	Yes
1.7	Solution of system of non-linear equations							
1.7.1	Direct Approach	Using simple substitution	N/A	Yes	No	2 unknowns		Yes
1.7.2	Newton-Raphson Method		Yes	Yes	Yes	2 unknowns	Direct Approach	Yes

Unit 2: Solution of System of Linear Algebraic Equations (8 hours)

Sub Unit		Further Details	Formula Derivation	Working Mechanism	Algorithm	Numerical	Comparison with	Advantages/ Limitations
2.1	Direct Methods	Overview	---	---	---	---	Iterative Methods	Yes
2.1.1	Gauss Jordan Method		N/A	Yes	Yes	3 and 4 unknowns	Gauss Elimination	Yes
2.1.2	Gauss Elimination Method	Partial and Complete Pivoting: Why and how	N/A	Yes	Yes	3 and 4 unknowns	Gauss Jordan	Yes
2.1.3	Matrix inverse using Gauss Jordan and elimination methods	Without pivoting	Yes	Yes	Yes	3×3 matrix	Each other	No
2.1.4	Factorization Methods	Do-Little's Method & Crout's Method	Yes	Yes	Yes	3×3 matrix or 3 unknowns	Each other	No
2.2	Iterative Methods	Overview	---	---	---	---	Direct Methods	Yes
2.2.1	Jacobi's Method		---	Yes	Yes	3 and 4 unknowns	Gauss-Seidal Method	Yes
2.2.2	Gauss Seidal Method		---	Yes	Yes	3 and 4 unknowns	Jacobi's Method	Yes
2.3	Determination of largest and smallest Eigen values and corresponding vectors using the power method	- Review of Eigen values and Eigen vectors and their physical interpretation - Importance of dominant Eigen values in realistic problems - Smallest Eigen value using inverse Power Method	---	Yes	Yes	3×3 matrix	Direct Method	Yes

Unit 3: Interpolation (9 hours)

[illegible]

Unit 4: Numerical Differentiation and Integration (6 hours)

Sub Unit		Further Details	Formula Derivation	Working Mechanism	Algorithm	Numerical	Comparison with	Advantages/ Limitations
4.1	Numerical Differentiation	Significance						
4.1.1	Differentiation using polynomial interpolation formulae for equally spaced intervals	Up to second order derivatives using Newton's Forward and Backward interpolation formulae	Yes	Yes	No	Yes	---	Yes
4.1.2	Local maxima and minima from equally spaced data	- By approximating the function using cubic polynomial such that the derivative reduces to quadratic - Using Newton's forward and backward difference interpolation formulae	Yes	Yes	No	Yes	---	Yes
4.2	Numerical Integration	Significance						
4.2.1	Newton Cote's general quadrature formula	Derivation using Newton's Forward interpolation formula	Yes	---	---	---	---	---
4.2.2	Trapezoidal Rule, Simpson's 1/3 and 3/8 rules, Boole's rule, Weddle's rule	Geometric interpretation	Trapezoidal & Simpson's Rules	Yes	Yes	Yes	---	---
4.2.3	Romberg Integration	Using Trapezoidal rule	No	Yes	No	Yes	---	---
4.2.4	Gauss-Legendre Integration	Up to 3-point formula only	No	Yes	Yes	Yes	---	---

Unit 5: Solution of Ordinary Differential Equations (8 Hours)

- Definition and significance of ODEs in modeling physical phenomena
- Comparison of numerical solution with analytical solution

Sub Unit		Further Details	Formula Derivation	Working Mechanism	Algorithm	Numerical	Comparison with	Advantages/ Limitations
5.1	Initial Value Problems	Overview	---	---	---	---	---	---
5.1.1	Solution of first order equations							
	Taylor's Series Method		---	Yes	---	Yes	---	---
	Euler's Method		Yes (Graphically & Analytically)	Yes	Yes	Yes	---	Yes
	Runge-Kutta second order (RK-2) method		No	Yes	Yes	Yes	---	Yes
	Runge-Kutta fourth order (RK-4) method		No	Yes	Yes	Yes	---	Yes
5.1.2	Solution of system of first order ODEs via Runge-Kutta methods	Using RK-1 (Euler), RK-2 and RK-4	No	Yes	Yes	2 dependent variables	---	---
5.1.3	Solution of second order ODEs via Runge-Kutta methods	• By reduction to a system of first order ODEs • Using RK-1 (Euler), RK-2 and RK-4	No	Yes	Yes	Yes	---	---
5.2	Two-point boundary value problems	Overview						
5.2.1	Shooting Method	Up to second order equations	---	Yes	Using RK-4 method	Using Euler's (RK-1) method	Finite Difference method	Yes
5.2.2	Finite Difference Method	Up to second order equations	Approximation of first and second order derivatives	Yes	Yes	Using 3 or 4 sub-intervals	Shooting method	Yes

Unit 6: Solution of Partial Differential Equations (7 hours)

Sub Unit		Further Details	Formula Derivation	Working Mechanism	Algorithm	Numerical	Comparison with	Advantages/ Limitations
6.1	Introduction and Classification	- Definition and role of PDEs in modeling physical phenomena - ODE Vs. PDE comparison - Analytical Vs. Numerical Solution - Standard forms of second order PDEs - Classification: Elliptic, Parabolic, Hyperbolic	---	---	---	---	---	---
6.2	Finite Difference approximations of partial derivatives	Approximation of first and second order partial derivatives	No	---	---	---	---	---
6.3	Solution of elliptic equations							
6.3.1	Laplace equation		Recurrence relation	Yes	Yes	using Gauss-Seidal Iteration	---	---
6.3.2	Poisson equation		Recurrence relation	Yes	Yes	using Gauss-Seidal Iteration	---	---
6.4	Solution of parabolic and hyperbolic equations							
6.4.1	One-dimensional heat equation							
	Bendre-Schmidt method		Recurrence relation	Yes	Yes	Yes	---	---
	Crank-Nicolson method		Recurrence relation	Yes	Yes	Yes	---	---
6.4.2	Wave Equation	Convergent case only	Recurrence relation	Yes	No	Yes	---	---

Evaluation:

Chapter	Hours	Marks*
1: Solution of Non-Linear Equations	7	10
2: Solution of System of Linear Algebraic Equations	8	10
3: Interpolation	9	10
4: Numerical Differentiation and Integration	6	10
5: Solution of Ordinary Differential Equations	8	10
6: Solution of Partial Differential Equations	7	10
Total	45	60

Coverage:

- There may be minor deviation in marks distribution (± 2)
- At least two questions (~ 10 marks) related to programming
- At least one theoretical question (~ 5 marks) (derivation, working mechanism, etc.)
- Generally, two diverse questions from each chapter

Practical (45 hours)

- Programming language to be used: Python
- Results to be visualized wherever possible
- Practical report contents:
Working Principle, Algorithm/Pseudocode, Source code, Test Cases

Unit 1: Basics of Programming in Python

- Basic data types
 - int, float, string, Boolean
 - type conversion of int and float from/to string
 - basic math and string operations
- Data Structures
 - Lists: basic list operations, using lists as arrays and matrices, generating sequence of numbers, list comprehension, map and zip function, aggregate functions
 - Tuples, Dictionaries, and Sets: introduction and simple examples
- Basic input / output
 - Formatted output
 - Conversion of input string into int / float and list
- Control Flow
 - if ... elif ... else
 - looping: while, for, break, and continue
 - raising exceptions and exception handling
- Functions and modules
 - construct user-defined functions using def and lambda expression
 - evaluating user-input string representation of mathematical functions
 - knowledge of packages and modules
- Basic numerical and scientific computation (using numpy and scipy)
 - Using numpy arrays
 - Library functions for solving Non-linear equations:
 - `scipy.optimize.fsolve`
 - `scipy.optimize.root`
 - `numpy.roots`

- Library/built-in functions for Linear Algebra:
 - `*`, `@` (matrix multiplication), `numpy.dot`,
 - `scipy.linalg.{solve, inv, eig, lu}`
- Library functions for Interpolation:
 - `scipy.interpolate.{lagrange, CubicSpline, interp1d}`
 - `scipy.optimize.curve_fit`, `numpy.polyfit`
- Library functions for Differentiation and Integration:
 - `numpy.gradient`
 - `numpy.trapz`
 - `scipy.integrate.{simps, quad, romberg, fixed_quad}`
- Library functions for Ordinary Differential Equations:
 - `scipy.integrate.solve_ivp`
 - `scipy.integrate.solve_bvp`
- Graphical Visualization:
 - Generate well annotated plots: scatter plot, function plots, heat map
 - `matplotlib.pyplot` functions:
 - `figure`, `show`, `savefig`
 - `plot`, `scatter`, `axhline`, `axvline`, `axline`
 - `title`, `xlabel`, `ylabel`, `text`
 - `legend`, `grid`
 - `xlim`, `ylim`
 - `imshow`, `colorbar`
 - attributes: `color`, `linestyle`, `linewidth`, `marker`, etc.

Units 2-7:

- Implement higher level algorithm by self
- Allow use of built-in functions for lower-level jobs
(example: min, max, sum, matrix multiplication, dot product, etc.)
- Visualize graphs or results wherever possible or beneficial